

Codex + Claude Code

The Dynamic Duo Field Guide

Stop picking sides. Use both.

by Mark Kashef
Early AI Dopters

What's inside

Every week somebody picks a new winner. Opus. Claude Code. Codex. The bickering never ends. But the actual play is using both as a team. This guide replays the entire video as a take-home reference. Every diagram, every command, every pattern, every plain-English explanation. Read it once before you build, keep it open while you build, screenshot the decision tree page and pin it next to your editor.

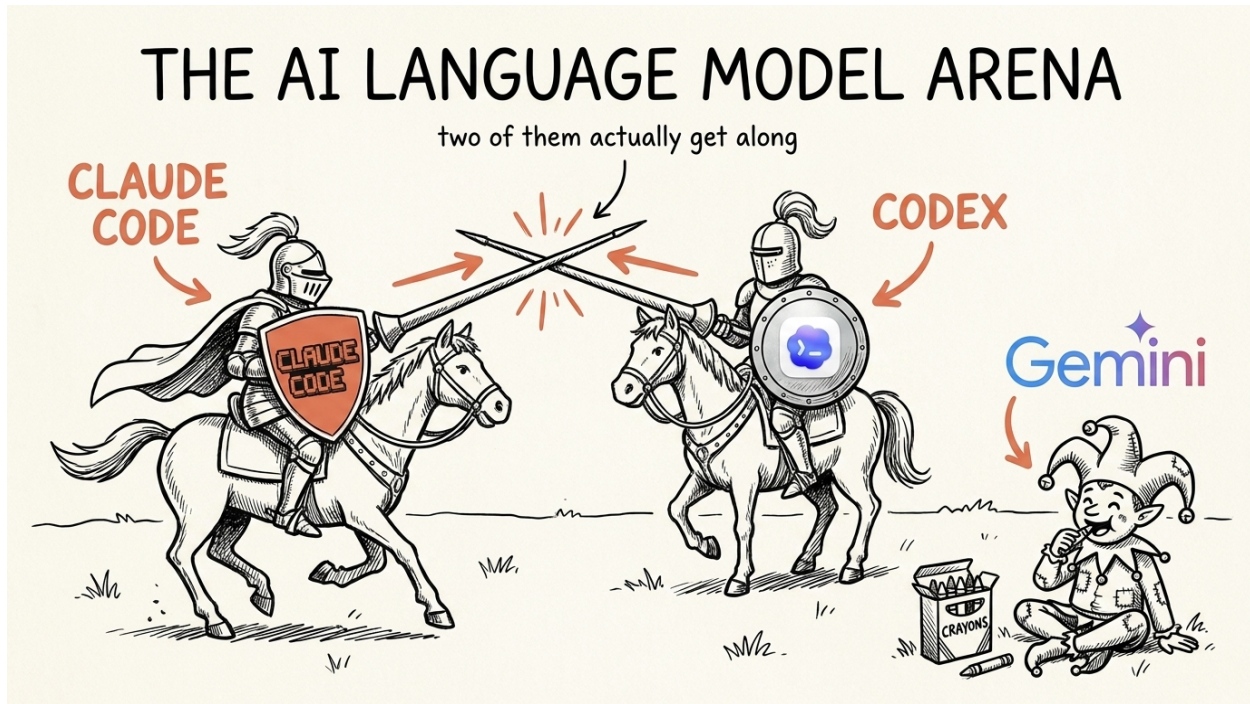
Contents

- 01** The Arena: Why Both, Not One
 - 02** What The Plugin Actually Is
 - 03** Background Mode And The Review Gate
 - 04** The Decision Tree
 - 05** `/codex:review`
 - 06** `/codex:adversarial-review`
 - 07** `/codex:rescue`
 - 08** `/codex:status + /codex:result`
 - 09** `/codex:setup --enable-review-gate`
 - 10** Pattern 1: Audit An Existing App
 - 11** Pattern 2: Plan A Feature With Codex Pushback
 - 12** Pattern 3: Background Rescue While You Build
 - 13** Pattern 4: Pre-Ship Stress Test
 - 14** Pattern 5: Auto-Review Gate (Autopilot)
 - 15** The \$120 Stack
- Cheat Sheet

01

The Arena

Why both, not one.



Right now, every conversation about AI coding tools online is treated like a sports rivalry. People pick a team. They argue about who is winning. They post benchmarks like fans posting box scores. The whole thing is exhausting, and it misses the actual point.

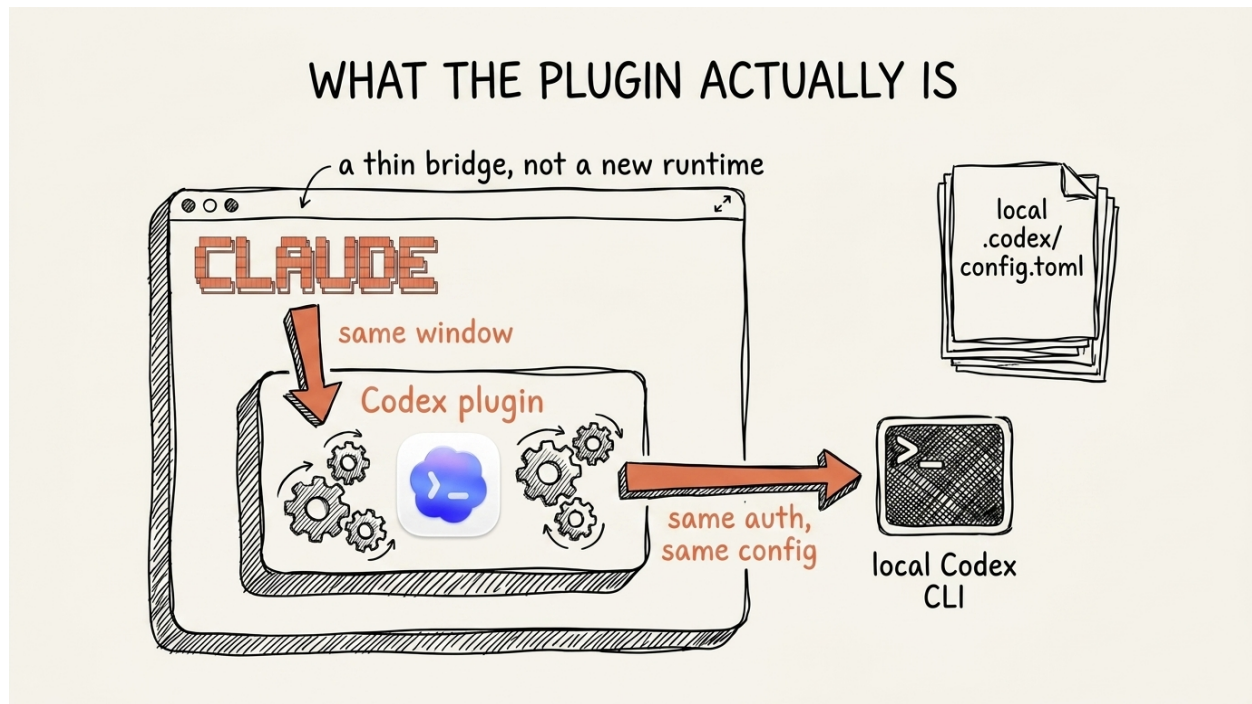
These are not interchangeable tools. **Claude Code** is great at planning what to build, writing copy, designing user-facing things, and big-picture thinking. **Codex** is built like a master code surgeon. It audits. It spots the kind of subtle bugs that ship to production and ruin somebody's Saturday. It plays devil's advocate on every design decision. **Gemini**, for code work specifically, is still finding its feet. It will catch up, but this guide is about Claude and Codex.

The frame

Treat them as a duo, not rivals. One plans. One double-checks. The plugin is the bridge that makes the partnership work without you switching tabs.

What The Plugin Actually Is

Not a new app. A bridge.



Quick sanity check before you use anything. The Codex plugin for Claude Code is not a brand-new app. It is not a separate runtime. It is not yet another login to manage.

Think of it like installing a Chrome extension. The extension does not replace Chrome. It just adds new capabilities to the browser you are already using. Same idea here. The plugin lives inside Claude Code and adds new commands you can type, all of which start with **/codex**:

Specifically, the plugin connects to your existing Codex CLI on your computer. CLI just means command-line tool, the kind of thing developers run in a terminal. If you are already paying for ChatGPT Plus, you already have access to Codex. You do not need a new API key. You do not need a new login. The same configuration you already have on your machine just works.

Install in 60 seconds:

```
/plugin marketplace add openai/codex-plugin-cc
```

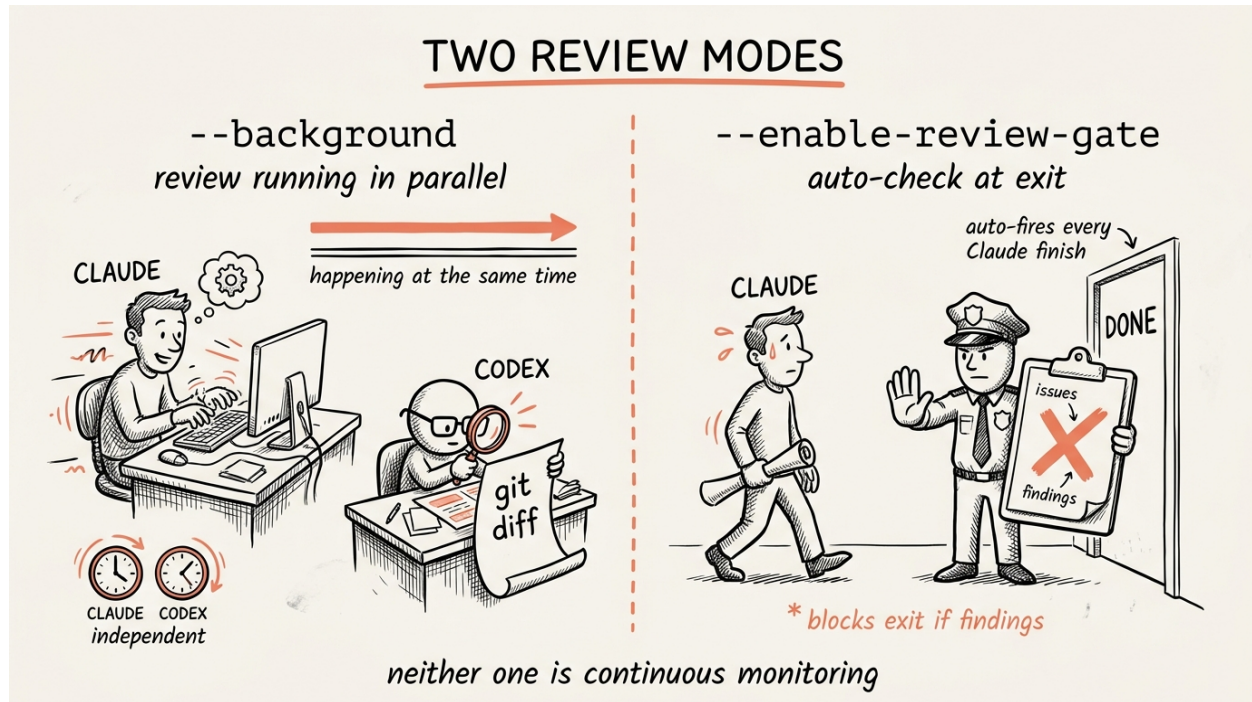
```
/plugin install codex@openai-codex
```

```
/reload-plugins
```


`/codex:setup`

Background Mode + Review Gate

Two flags that sound continuous. Neither one actually is.



--background: the parallel review

When you put this flag at the end of any Codex command, Claude Code spawns a separate process to run the review. You get a tracking ID back right away. Claude does not wait. You keep working. Codex chews on the change in parallel, which means at the same time as you. When Codex is done, the result is just sitting there waiting for you. You check it whenever you are ready.

Without this flag, the review runs in the foreground, which means Claude blocks and you stare at a spinner until Codex finishes. That is fine for tiny changes. Brutal for anything bigger.

--enable-review-gate: the security guard at the exit

Completely different mechanism. When you flip this on, Claude Code installs what is called a **Stop hook**. A hook is a built-in feature that fires automatically at a specific moment. The Stop hook fires only when Claude tries to finish a turn. Not every keystroke. Not every file save. Just at the moment Claude says, okay, I am done with this task.

At that moment, the hook checks if the turn touched code. If it did, the hook automatically runs Codex on the new work. If Codex finds something material, Claude is blocked from finishing. Codex's findings go back to Claude. Claude has to address them before it can call the task done.

Easiest way to think about it

The review gate is a security guard at the exit. Not a guard following you around. Only checks you on the way out the door.

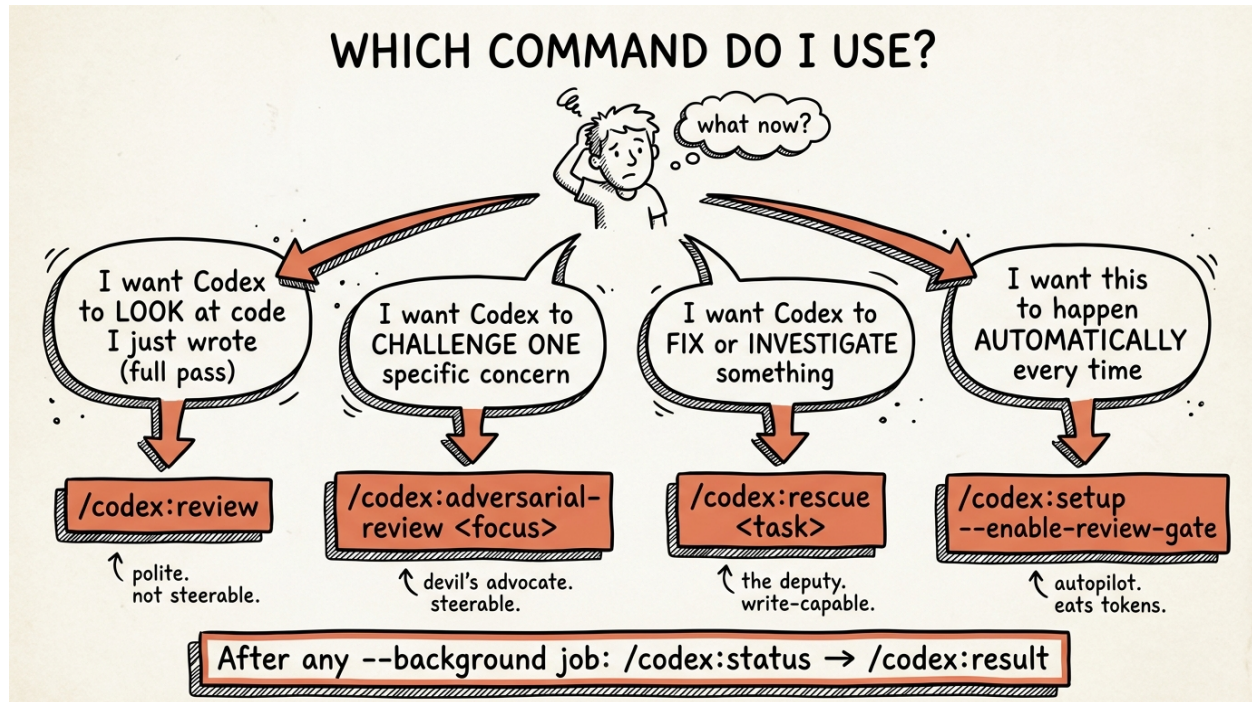
Cost reality

--background is one Codex review per invocation. Cheap. Predictable.

The review gate is one review every *time* Claude tries to finish a turn that touched code. If Claude has five back-and-forths in a session, that is five reviews. Adds up fast. Only flip the gate on for focused sessions.

The Decision Tree

Memorize this one slide.



There are seven slash commands the plugin gives you, but really you only ever make four decisions.

Decision 1. You want Codex to LOOK at the code Claude just wrote. Full pass, surface anything noteworthy. `/codex:review`

Decision 2. You have ONE specific concern (auth, data loss, timezones).
`/codex:adversarial-review` with focus text

Decision 3. You want Codex to actually FIX or INVESTIGATE something. Take it off your plate.
`/codex:rescue` with a task

Decision 4. You want this to happen automatically every time Claude finishes a turn. `/codex:setup --enable-review-gate`

The two helpers, `/codex:status` and `/codex:result`, are just for checking on background jobs. The rest is plumbing.

/codex:review

The polite second opinion.

/codex:review

the polite second opinion



- *Use it when:* you just had Claude write code and want a full review pass
- *Custom text after?* **NO** -- flags only, you cannot add focus text
- *Sample:* `/codex:review --background`

read-only, never changes code

When you run `/codex:review`, Codex looks at every file you have changed and gives you a comprehensive review. Bugs it spots. Missing edge cases. Quality concerns. Security issues that map to the OWASP top ten, which is just a famous list of common web vulnerabilities. The full picture.

This command is **NOT steerable**. You cannot tell it what to focus on. You cannot type `/codex:review focus on auth security`. The command ignores anything you type after it. It always does the full pass. If you want a focused review on one specific concern, use the next command.

Use it when Claude just finished a chunk of work and you want a regular code review pass. Add `--background` if the change is big and you do not want to wait.

Sample command:

```
/codex:review --background
```


/codex:adversarial-review

The killer feature. The devil's advocate.

/codex:adversarial-review

the devil's advocate



- **Use it when:** you want to challenge **ONE** specific concern (auth, data loss, design)
- **Custom text after?** **YES** -- add focus text after the flags
- **Sample:** /codex:adversarial-review challenge timezone handling

steerable, read-only, the killer feature

This is steerable. You add focus text after the flags and Codex grills exactly that concern. Things like 'challenge whether the caching strategy handles partial failures.' Or 'pressure test what happens if two users hit this at the same time.' Or 'be paranoid about data loss.'

Codex then plays bad cop on that ONE concern aggressively. It questions assumptions. It looks for edge cases. It asks the kind of questions a senior engineer would ask at code review time. It is told, in its own internal prompt, to try and break confidence in the change. To find reasons it should NOT ship. That stance is what makes it different from /codex:review.

Like the regular review command, this one is read-only. Codex does not edit your code. It delivers findings.

Use it when something specific scares you about a change. Auth flows. Data migrations. Payment paths. Anywhere a mistake costs you real money or real trust.

Sample command:

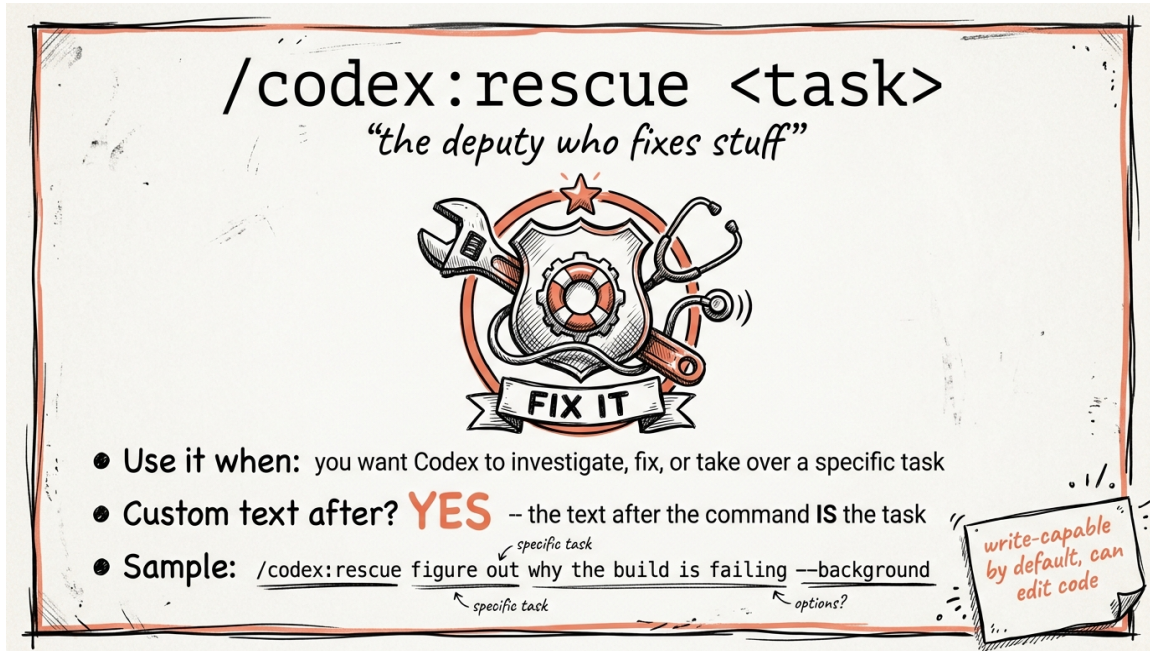
```
/codex:adversarial-review challenge race conditions
```

Heads up

This is the command most people don't realize exists. It's the single biggest unlock vs solo Claude.

/codex:rescue

The deputy. The command for delegation.



Slash codex rescue is the one you use when you want Codex to actually take work off your plate. The text you type after the command IS the task itself. Like `/codex:rescue figure out why the build is failing in CI` or `/codex:rescue refactor this database connection to be more resilient.`

Unlike the review commands, rescue is **write-capable** by default. Meaning Codex can actually edit your code. If you want it to just diagnose without touching the code, say so explicitly in the task text. Like 'do not change anything yet, just diagnose.'

Use it when you want Codex to investigate something, propose a fix, or just go solve a problem while you stay focused on something else. Always pair it with `--background` for non-trivial tasks so you do not have to wait.

Sample command:

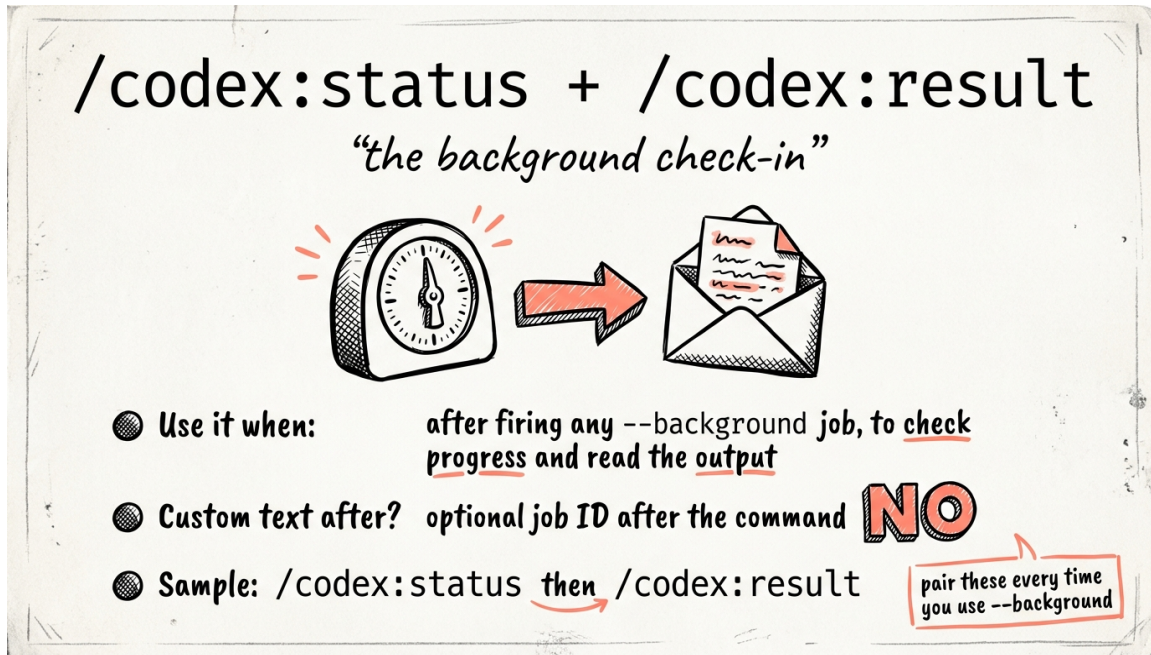
```
/codex:rescue figure out why the build is failing --background
```

Heads up

Common confusion: rescue is for taking work off your plate. If you just want a read-only opinion, use `/codex:review` or `/codex:adversarial-review` instead.

/codex:status + /codex:result

The background check-in. Plumbing.



These two are utility commands. Pair them every time you fire something with `--background`.

/codex:status shows you a list of running and recently finished Codex jobs in this project. Like checking the laundry timer. You can see if a job is still running, finished, or failed.

/codex:result shows you the actual output of a finished job. The findings, the diagnosis, the recommended fix. Whatever Codex came back with.

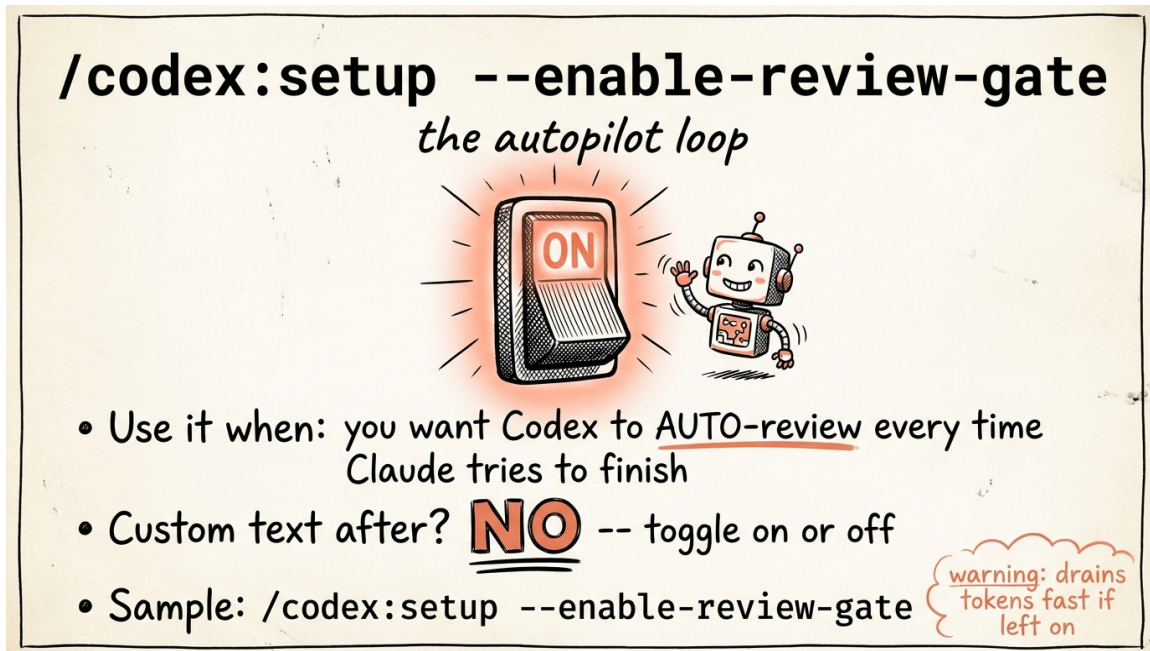
There is also **/codex:cancel**, which kills a running job if you change your mind.

Sample command:

```
/codex:status /codex:result
```


/codex:setup --enable-review-gate

The autopilot toggle.



This flips on the Stop hook that automatically runs Codex every time Claude tries to finish a turn that touched code. If Codex finds something material, Claude is blocked from finishing and has to address the finding first. If Codex is happy, Claude gets through.

The flag is on or off. There is no middle ground. To turn it off, run `/codex:setup --disable-review-gate`.

Important warning: this drains tokens fast because every Claude finish triggers another review. If you are deep in a build and Claude is finishing turns every minute or two, you can run through your monthly token allowance quickly.

Recommendation: only flip it on for focused sessions where you really want the safety net. Flip it off when you are done. Do not leave it on overnight.

Sample command:

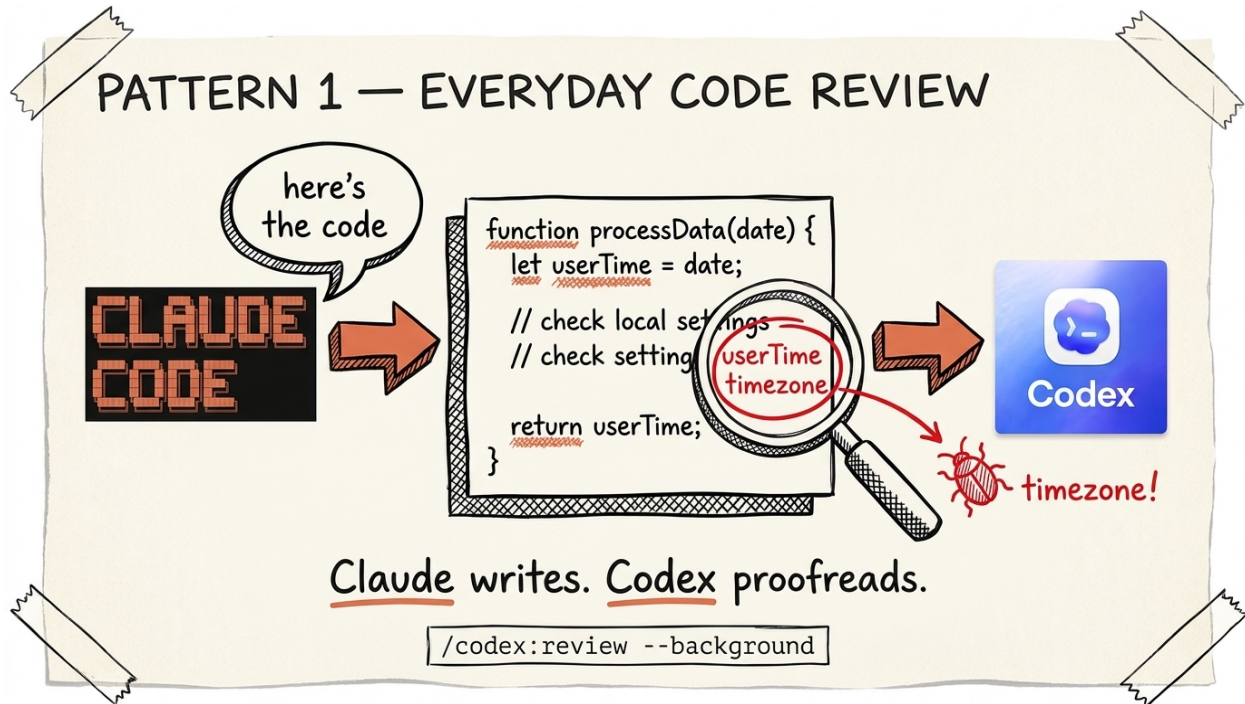
```
/codex:setup --enable-review-gate
```


Heads up

Pair with `/codex:setup --disable-review-gate` when you are done. Or your tokens will not survive the month.

Pattern 1: Audit An Existing App

Claude wrote it. Codex audits.



The video uses a small URL shortener called Linkboard. Paste a URL, get a short link, click it, the click counter goes up. Standard side-project app.

The setup is real and relatable: built it weeks ago, it works, never had a senior engineer audit it. Exactly the situation `/codex:review` was made for.

What Codex finds (typical):

- **Slug collision risk.** The function that creates short link codes uses `Math.random`, which can produce duplicates and crash with a 500 error.
- **No URL validation.** A malicious person could submit a javascript: URL that runs code in someone's browser when they click the link.
- **Click counter race condition.** Two clicks at the exact same instant can lose one of the counts. A fancy way of saying: the counter math is not safe under load.
- **No pagination on the link list.** If you had a hundred thousand links, the page would melt trying to load them all at once.

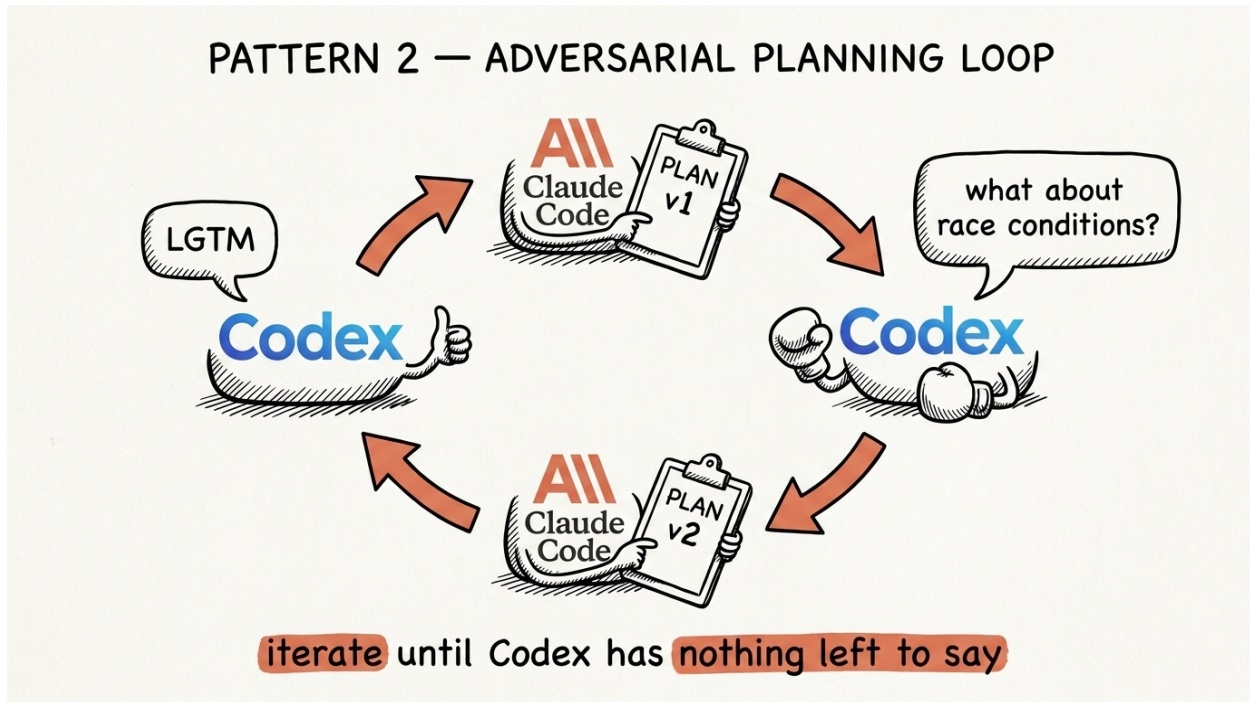
The takeaway

These are not stylistic nits. Real bugs. Living in code that has been running for weeks. Codex found them in 90 seconds. That is the duo working as a duo.

11

Pattern 2: Plan A Feature With Codex Pushback

Iterate until Codex has nothing to say.



Most people just write code. The point of this pattern is to NOT do that. Plan first. Then have Codex stress-test the plan before any code is written.

The loop:

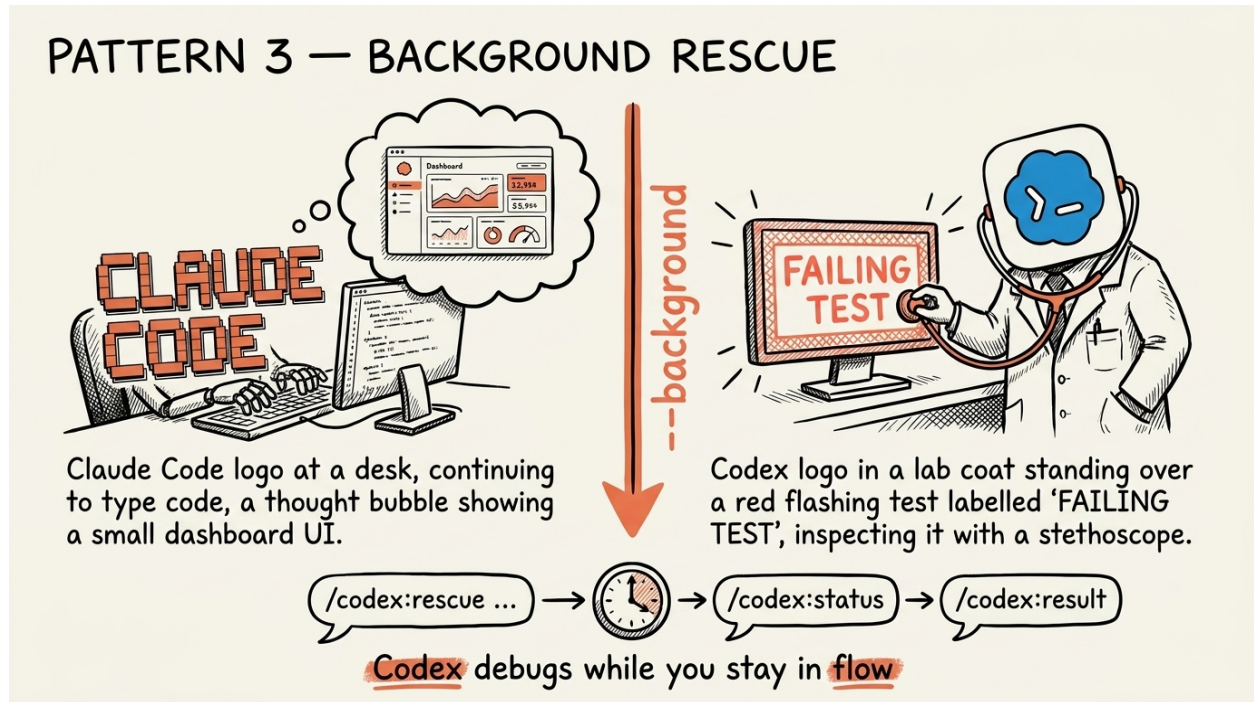
- Ask Claude for a plan only. No code. Numbered list. Cover edge cases, time zones, what happens when something fails halfway.
- Send the plan to Codex with `/codex:adversarial-review` and pressure-test text. What breaks if two people use it at once? What if someone clicks twice? Where could this fall apart?
- Codex returns design-level critiques. Not code review, idea review.
- Feed Codex's pushback back to Claude. Claude reflects, revises, outputs a new plan.
- Optional: one more adversarial round. Stop when Codex has no substantive findings.

Why this is the most valuable pattern

Two rounds of plan critique and the design is bulletproof. You have not written a single line of code. This is the move most people skip and the one that compounds the most over time.

Pattern 3: Background Rescue While You Build

Codex debugs while you stay in flow.



Tests are little automated checks that run code and confirm it does what you expect. When a test fails, most people stop everything and dig in. That is fine, but it can derail an entire afternoon.

Instead, hand the failing test to Codex with `/codex:rescue`. The `--background` flag means Codex starts working on it in parallel. You keep building with Claude on something else.

Sequence:

- Run the tests. See what fails.
- `/codex:rescue` figure out why X test is failing and tell me the smallest fix `--background`
- While Codex digs, ask Claude for the next feature.
- Check `/codex:status`. When done, `/codex:result`.
- Apply Codex's fix via Claude. Rerun tests.

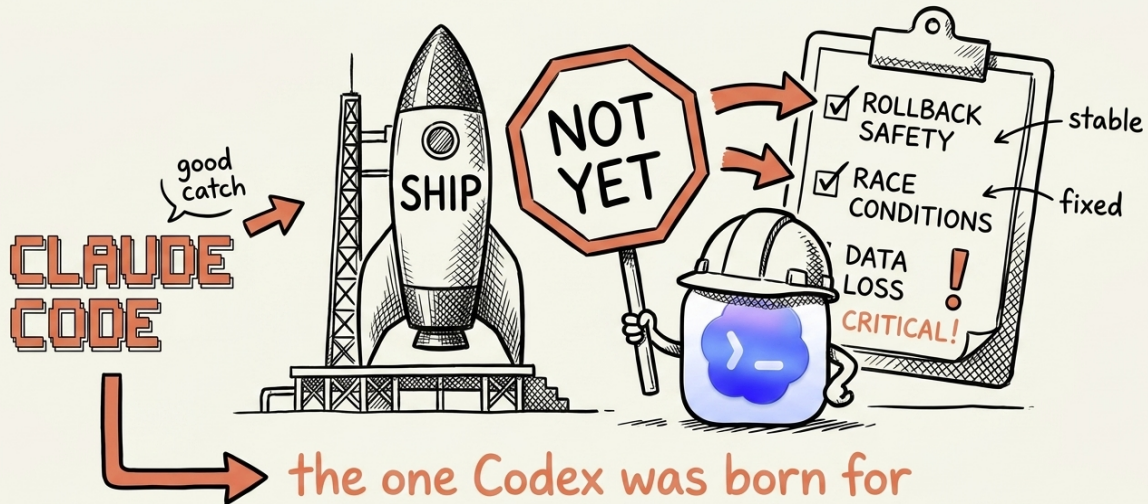
The unlock

Codex was debugging in a side thread the whole time. You never dropped flow. That is the pattern that day-to-day saves the most time.

Pattern 4: Pre-Ship Stress Test

The high-stakes safety net before risky changes.

PATTERN 4 — PRE-SHIP STRESS TEST



`/codex:adversarial-review` challenge rollback and race conditions

Some changes are dangerous. A delete-all button. An auth migration. A payment flow change. One wrong click and someone loses real data or real money.

The right move is to play it safe by default. For deletes, that means a **soft delete** by default. Soft delete just means the records are flagged as deleted but not actually erased. The real, permanent delete is gated behind an explicit force-delete option.

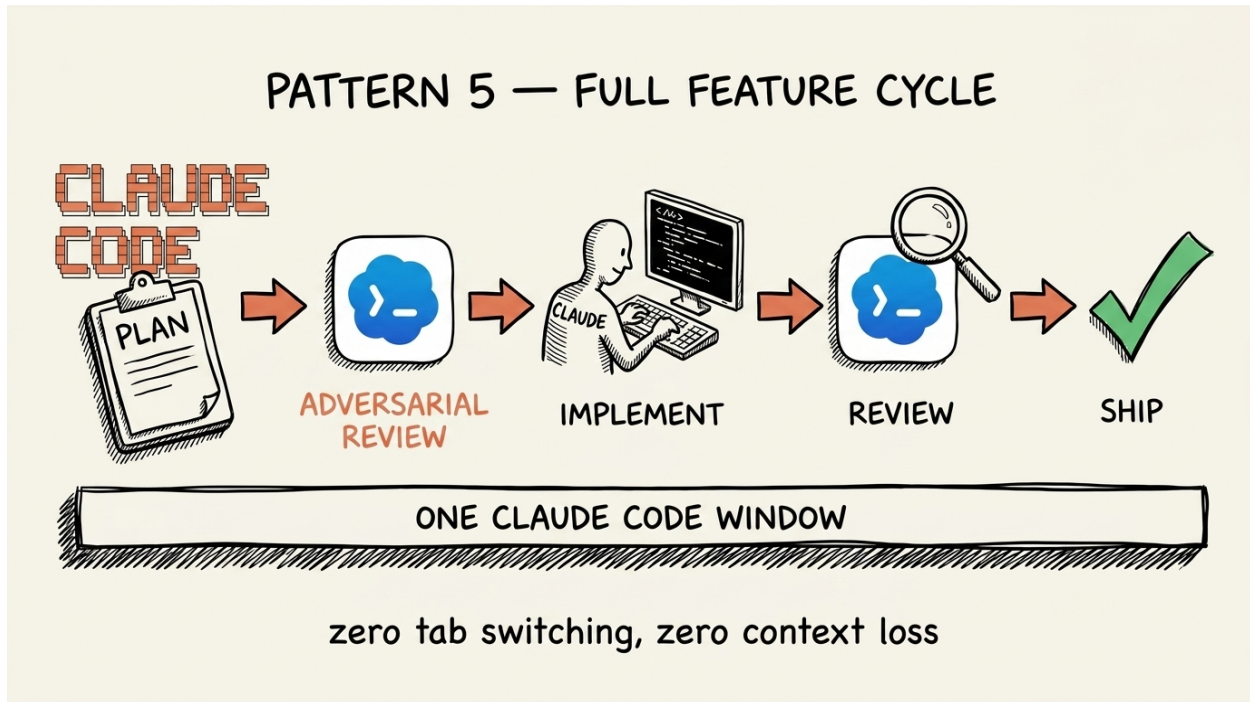
Once Claude builds it, do NOT just commit. Ask Codex to be paranoid. `/codex:adversarial-review` with a paranoid focus: 'be paranoid, what could go wrong here, are deletes respected everywhere we read this data, what happens if the user hits the button twice, does the force option need a safety check.'

The would-I-actually-ship moment

Codex returns the kind of senior engineer questions you'd get at a real code review by a real human teammate. Now you don't have to skip the review just because there is no team.

Pattern 5: Auto-Review Gate (Autopilot)

Claude and Codex bounce the work back and forth without you typing.



This is the closest the plugin gets to autonomous mode. You flip the gate on, give Claude a small but tricky feature, and watch it loop:

- `/codex:setup --enable-review-gate`
- Claude builds, tries to call it done.
- Stop hook fires. Codex reviews.
- If Codex finds something, Claude is blocked. Findings come back. Claude addresses them.
- Tries to finish again. Hook fires again. Eventually Codex approves and the turn ends.
- `/codex:setup --disable-review-gate (DO NOT FORGET)`

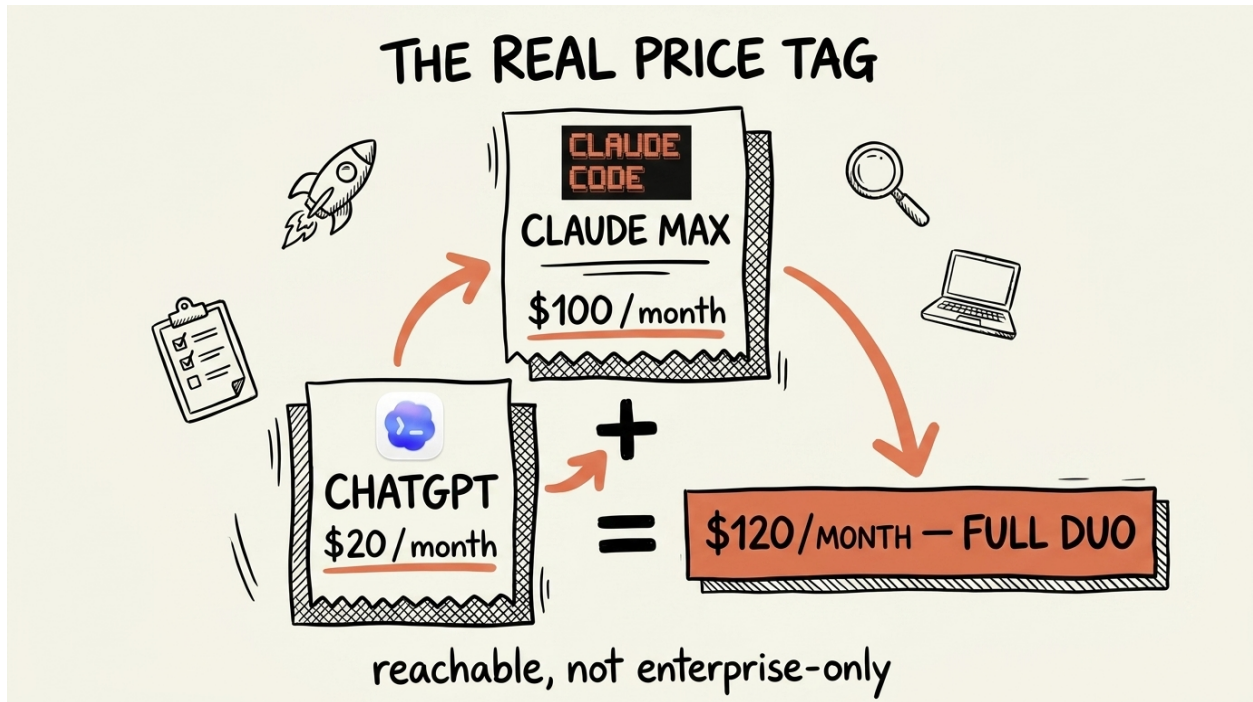
The most important step

Turn the gate back off when you are done. If you forget, every Claude finish triggers a Codex review and you will burn through your monthly tokens fast.

15

The \$120 Stack

What this actually costs.



Claude Max is the \$100/month tier. That gets you Claude Code with generous usage limits.

ChatGPT Plus is \$20/month. That gets you Codex.

\$120/month total gets you the entire setup. No enterprise contract. No API keys to manage. No billing surprises. Two consumer subscriptions and you are building with the same stack the top engineers are using right now.

The closing message

Stop picking sides. Claude plans. Codex audits. They are teammates, not rivals. The bickering about which model wins this week is the wrong fight. The right fight is using both for what each one is genuinely good at.

Cheat Sheet

Polite full code review

Read-only, not steerable. Use after Claude writes code.

```
/codex:review --background
```

Stress-test ONE concern

Steerable. Add focus text after the flags.

```
/codex:adversarial-review challenge race conditions
```

Delegate a task

Write-capable. The text after the command IS the task.

```
/codex:rescue figure out why the build is failing --background
```

Check on a background job

Pair with /codex:result.

```
/codex:status
```

Read the result

Shows the full output of a finished job.

```
/codex:result
```

Kill a running job

Stops a background Codex job.

```
/codex:cancel
```

Turn on auto-review

Codex blocks Claude's finish until findings are addressed.

```
/codex:setup --enable-review-gate
```

Turn off auto-review

DO NOT forget this at the end of a session.

```
/codex:setup --disable-review-gate
```

Check Codex install

Confirms Codex is installed and authenticated.

```
/codex:setup
```

Want the deeper teaching? Read the Concepts Companion that came in this kit. Want to be part of the conversation? Join Early AI Dopters.

Early AI Dopters